Primitiver Typ	Wrapper-Typ
`byte`	`Byte`
`short`	`Short`
`int`	`Integer`
`long`	`Long`
`float`	`Float`
`double`	`Double`
`char`	`Character`
`boolean`	`Boolean`

B.5 Cast-Operator für Objekttypen

Da viele Objekte Teil einer Vererbungshierarchie von Objekttypen sind, ist es manchmal notwendig, die Objektreferenz eines Datentyps in eine Referenz eines Subtyps tiefer in der Vererbungshierarchie umzuwandeln. Dieser Prozess wird auch als *Casting* (oder *Downcasting*) bezeichnet. Der zugehörige Cast-Operator besteht aus dem in runden Klammern gesetzten Namen des Zieltyps (eine Klasse oder ein Interface) und steht vor einer Variablen oder einem Ausdruck. Zum Beispiel

```
Auto a = (Auto) kfz;
```

Wenn der deklarierte (d.h. statische) Typ der Variablen **`kfz`** vom Typ **`Kraftfahrzeug`** ist und **`Auto`** eine Subklasse von **`Kraftfahrzeug`** ist, wird sich diese Anweisung kompilieren lassen. Zur Laufzeit wird zusätzlich geprüft, ob das Objekt, auf das sich **`kfz`** bezieht, wirklich ein **`Auto`** ist und nicht eine Instanz eines anderen Subtyps.

Beachten Sie unbedingt, dass das Casting zwischen Objekttypen etwas völlig anderes ist als das Casting zwischen primitiven Typen (Anhang B.2). Insbesondere bewirkt das Casting zwischen Objekttypen keine Änderung des betreffenden Objekts. Es geht allein darum, Zugriff auf Typinformationen zu erhalten, die bereits Teil des Objekts sind, weil sie zu seinem dynamischen Typ gehören.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

ANHANG

C Operatoren

C.1 Arithmetische Ausdrücke

Java bietet zahlreiche Operatoren für arithmetische und logische Ausdrücke. Tabelle C.1 zeigt alle Konstrukte, die als Operatoren eingestuft werden, einschließlich dem Cast-Operator und der Parameterübergabe. Bei den meisten Operatoren handelt es sich entweder um binäre Operatoren (deren Operanden links und rechts stehen) oder um unäre Operatoren (die nur einen Operanden haben). Die wichtigsten binären arithmetischen Operatoren lauten:

- `+` Addition
- `-` Subtraktion
- `*` Multiplikation
- `/` Division
- `%` Modulo oder ganzzahliger Rest bei Division

Die Ergebnisse der Operationen Division und Modulo hängen davon ab, ob ihre Operanden ganze Zahlen oder Fließkommazahlen sind. Bei einer Division zwischen zwei `int`-Werten wird ein `int`-Ergebnis ohne Rest geliefert, aber zwischen Fließkommawerten wird ein Fließkommawert geliefert:

- `5 / 3` liefert als Ergebnis `1`
- `5.0 / 3` liefert als Ergebnis `1.66666666666666667`

(Beachten Sie, dass nur einer der Operanden eine Fließkommazahl sein muss, um zu einem Fließkommaergebnis zu führen.)

Wenn in einem arithmetischen Ausdruck mehr als ein Operator enthalten ist, werden für die Auswertungsreihenfolge *Prioritätsregeln* für Operatoren angewendet. In Tabelle C.1 erscheinen Operatoren mit höherer Priorität weiter oben; wir sehen also beispielsweise, dass Multiplikation, Division und Modulo eine höhere Priorität haben als die Addition und die Subtraktion. Die folgenden Ausdrücke liefern deshalb beide das Ergebnis `100`:

```
51 * 3 - 53
154 - 2 * 27
```

Binäre Operatoren mit derselben Priorität werden von links nach rechts ausgewertet, während unäre Operatoren mit derselben Priorität von rechts nach links ausgewertet werden.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

Wenn es notwendig wird, die Auswertungsreihenfolge zu ändern, können Klammern verwendet werden. So lautet das Ergebnis der beiden folgenden Beispiele **100**:

```
(205 - 5) / 2
2 * (47 + 3)
```

Die wichtigsten unären Operatoren sind `-`, `|`, `++`, `--`, `[]` und `new`. Sicher ist Ihnen aufgefallen, dass `++` und `--` in den beiden obersten Reihen der Tabelle C.1 aufgeführt sind. Die Operatoren in der obersten Reihe haben ihren Operanden zur Linken und die in der zweiten Reihe zur Rechten.

Tabelle C.1
Java-Operatoren, höchste Priorität oben.

`[]`	`.`	`++`	`--`	(*Parameterliste*)							
`++`	`--`	`+`	`-`	`!`	`~`						
`new`	(*Cast*)										
`*`	`/`	`%`									
`+`	`-`										
`<<`	`>>`	`>>>`									
`<`	`>`	`>=`	`<=`	`instanceof`							
`==`	`!=`										
`&`											
`^`											
`|`											
`&&`											
`||`											
`?:`											
`=`	`+=`	`-=`	`*=`	`/=`	`%=`	`>>=`	`<<=`	`>>>=`	`&=`	`|=`	`^=`

C.2 Boolesche Ausdrücke

In booleschen Ausdrücken werden Operatoren auf Operanden angewendet, um entweder den Wert `false` oder den Wert `true` zu liefern. Solche Ausdrücke werden oft in bedingten Anweisungen für die Bedingung und in Schleifen für die Schleifenbedingung verwendet.

Die Vergleichsoperatoren kombinieren üblicherweise ein Paar arithmetische Operanden. Vergleichsoperatoren in Java sind:

`==` gleich-zu	`>` größer-als
`!=` nicht-gleich-zu	`<=` kleiner-gleich
`<` kleiner-als	`>=` größer-gleich

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

Die binären logischen Operatoren kombinieren zwei boolesche Ausdrücke, um ebenfalls einen booleschen Wert zu liefern. Die Operatoren sind:

`&&` und

`||` oder

`^` exklusiv-oder

Zusätzlich kann mit dem Operator

`!` nicht

ein boolescher Ausdruck von `true` in `false` umgewandelt werden oder umgekehrt.

C.3 Abkürzungsoperatoren

Sowohl `&&` als auch `||` werden auf ungewöhnliche Weise ausgewertet. Wenn der linke Operand von `&&` den Wert `false` hat, dann ist der Wert des rechten Operanden irrelevant und wird nicht ausgewertet. In ähnlicher Weise wird der rechte Operand von `||` nicht ausgewertet, wenn der linke Operand den Wert `true` hat. Sie sind deshalb auch als Abkürzungsoperatoren (*short circuit operators*) bekannt.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

ANHANG

D

Kontrollstrukturen in Java

D.1 Kontrollstrukturen

Kontrollstrukturen regeln die Abfolge, in der Anweisungen ausgeführt werden. Es gibt im Wesentlichen zwei Kategorien von Kontrollstrukturen: *Auswahlanweisungen* und *Schleifen*.

Eine Auswahlanweisung ist wie eine Gabelung, an der eine Entscheidung getroffen werden muss, welchem Weg durch den Rumpf der Methode (oder des Konstruktors) zu folgen ist. Eine *if-else-Anweisung* verlangt eine Entscheidung zwischen zwei Anweisungsblöcken, während eine *switch-Anweisung* mehrere alternative Anweisungsblöcke zur Auswahl anbietet.

Schleifen ermöglichen die wiederholte Ausführung eines Anweisungsblocks. Die Anzahl der Wiederholungen kann vorgegeben oder undefiniert sein. Für die erste Variante werden typischerweise *for-each*- oder *for-Schleifen* eingesetzt, die zweite Variante wird meist mit *while*- oder *do-Schleifen* realisiert.

Diese Charakterisierungen sind allerdings nicht verbindlich, Abweichungen sind in der Praxis häufig zu finden. Beispielsweise kann man die `if-else`-Anweisung auch zur Auswahl zwischen mehreren Alternativen nutzen, wenn der `else`-Teil eine untergeordnete (verschachtelte) `if-else`-Anweisung enthält, und eine `for`-Schleife kann auch für eine nicht festgelegte Anzahl von Wiederholungen eingesetzt werden.

D.2 Auswahlanweisungen

D.2.1 if-else

Die *if-else-Anweisung* gibt es in zwei Varianten, die beide durch einen booleschen Ausdruck gesteuert werden.

```
if (Ausdruck {
   Anweisungen
}
```

```
if (Ausdruck) {
   Anweisungen
}
else {
   Anweisungen
}
```

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr